

LLDD BE - Job 7 SyncCompetitorToDocument

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจสอบ และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว
Java เดิม	ภาคผนวกท้ายฉบับระบุ source file, ช่วงบรรทัด และ code Java เดิมสำหรับตรวจเทียบ behavior ก่อนย้ายระบบ

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. Overview

รายการ	รายละเอียด
Track	BE
Estimate	13 ชั่วโมง = implementation 10 + unit test 3 (30%)
Owner	Peerakorn <Pete> Sakunkaewphithak
Target repository	SBP/srm-sps-spsap-store-backend (NestJS + TypeORM · schema sps_store) — batch runner ผัง backend ไม่ผ่าน BFF · cron/พารามิเตอร์อยู่ใน backend config (env/config file)
Objective	บันทึกข้อมูลคู่แข่งเข้าเอกสาร: อ่านข้อมูลคู่แข่งล่าสุดจาก fgi_impact_competitors แล้วบันทึกเข้า document_competitors ผ่าน Document Service โดยตรง แทนการเขียนไฟล์ BPM06003O และ SFTP ไป BPM

Common contract reference: ทุกหัวข้อ API/FE ต้องยึด LLDD-BE-API-Common-Contracts.pdf และ LLDD-FE-Integration-Contracts.pdf สำหรับ error/auth/format/pagination/action/RBAC ก่อนลงรายละเอียดเฉพาะหน้าหรือเฉพาะ endpoint

2. Screen / Functional Scope

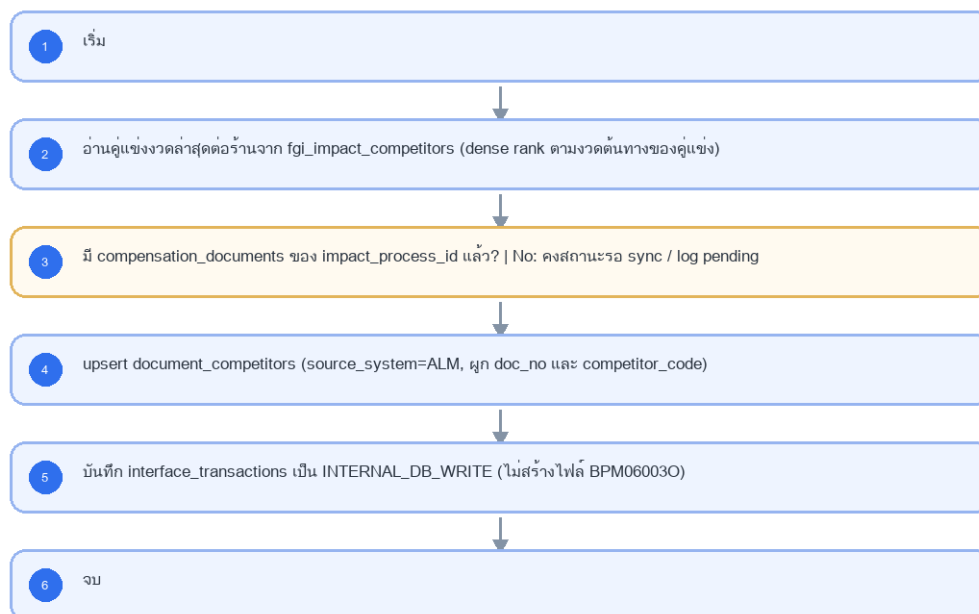
- Main class/script: document.service.syncCompetitors / (internal scheduler / service)
- Phase: B
- Output: document_competitors (DB)
- Estimate: 10 ชั่วโมง
- พารามิเตอร์/cron อ่านจาก backend config (config file/env) — ไม่มีตาราง job_configs และไม่มีหน้าจอบทความ (หน้า Flow Batch Job ในกลุ่มเมนู Flow เหลือแค่ Flowchart + Database ที่ใช้ · 2026-08-06)
- Runbook, rerun rule, risk และ history ตามเอกสาร Batch v4.0 · ผลการรันเขียน application log แบบ structured

3. Screenshot Reference

ไม่มีภาพหน้าจอสำหรับหัวข้อนี้ — เป็นเอกสารฝั่ง Backend/Batch ที่ไม่มี UI (ภาพหน้าจอทั้งหมดอยู่ในเอกสารชุด FE)

4. Implementation Flow Diagram (Reference)

LLDD BE - Job 7 SyncCompetitorToDocument



รูปที่ 1: Implementation flow reference: LLDD BE - Job 7 SyncCompetitorToDocument

5. Field, Format, and Validation

Field / UI	Format	Validation	Behavior
กำหนดการรัน (Cron)	30 17 7-31 * *	แก้ไขได้	ใช้รอบเดิม แต่ปลายทางเป็น DB ภายใน
Target table	document_competitors	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	upsert ด้วย doc_no / competitor_code / source_system=ALM
เงื่อนไขเลือกข้อมูล	งวดคู่แข่งล่าสุดต่อร้าน + forecast เริ่มต้น + ยังไม่ sync	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	คง business rule เดิม

5.9 Input / Progress / Output Contract

Stage	Contract for implementation
Input	FGI_IMPACT_COMPETITOR rows linked to active impact-process records and BPM/export confirmation state.
Progress	query latest competitor rows, skip already-confirmed transactions, create outbound payload per competitor, upload/export, insert confirm-receive rows.
Output	Competitor sync payload/output for downstream workflow; confirm-receive rows prevent duplicate export.

5.90 Job 7 Execution Stages

query latest competitor rows, skip already-confirmed transactions, create outbound payload per competitor, upload/export, insert confirm-receive rows.

Order	Service step	Repository	Output / failure contract
1	loadLatestDocumentCompetitors	documentCompetitorRepository	คืน metrics และ throw typed error; transaction/rerun ใช้ contract ด้านล่าง

Order	Service step	Repository	Output / failure contract
2	upsertDocumentCompetitors	documentCompetitorRepository	คืน metrics และ throw typed error; transaction/rerun ใช้ contract ด้านล่าง
3	recordInternalCompetitorSync	documentCompetitorRepository	คืน metrics และ throw typed error; transaction/rerun ใช้ contract ด้านล่าง
4	reconcileDocumentCompetitors	documentCompetitorRepository	คืน metrics และ throw typed error; transaction/rerun ใช้ contract ด้านล่าง

5.91 Job 7 Run Evidence

Evidence	Job-specific value	Acceptance
Input identity	FGI_IMPACT_COMPETITOR rows linked to active impact-process records and BPM/export confirmation state.	snapshot input file/business key/period in run record
Output identity	Competitor sync payload/output for downstream workflow; confirm-receive rows prevent duplicate export.	reconcile input, success, reject and skipped counts
Dedup proof	UNIQUE(doc_no,competitor_code); upsert และ prune เฉพาะ source_system=ALLMAP ให้ target ตรง source ปัจจุบันโดยไม่ลบแถว USER	rerun fixture produces no duplicate target business key
Transaction proof	upsert + prune document_competitors และ tracking INTERNAL_DB_WRITE ใน transaction เดียวต่อ doc_no	injected failure leaves no partial committed state outside documented boundary
Security proof	service account ภายในมีสิทธิ์ SELECT source และ INSERT/UPDATE target เท่านั้น; ไม่มี external credential	config/log/error contains no plaintext secret

5.92 Legacy Java Source Reference

Legacy file	Line range	Responsibility to carry forward
fcsJar/src/th/co/gosoft/fgi/main/ExportCompetitor.java	9-20	Legacy main entrypoint for competitor export.
fcsJar/src/th/co/gosoft/fgi/controller/ExportController.java	659-760	Query competitor data, generate file content, upload, backup, notification.
fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java	1596-1628	Query latest competitor rows eligible for export.

Line ranges refer to the legacy Java implementation under /Users/bank_mac/gosoft/java/SBP/fcsJar. Use these ranges to preserve business behavior while implementing the target Node job.

5.93 Target Repository and SQL Contract

Contract	Target implementation
Repository	documentCompetitorRepository
Idempotency / dedup	UNIQUE(doc_no,competitor_code); upsert และ prune เฉพาะ source_system=ALLMAP ให้ target ตรง source ปัจจุบันโดยไม่ลบแถว USER
Transaction boundary	upsert + prune document_competitors และ tracking INTERNAL_DB_WRITE ใน transaction เดียวต่อ doc_no
Security	service account ภายในมีสิทธิ์ SELECT source และ INSERT/UPDATE target เท่านั้น; ไม่มี external credential

Input / candidate query

```
SELECT d.doc_no, c.competitor_code, c.name_th, c.branch_th, c.opened_date, c.closed_date
FROM fgi_impact_competitors c
JOIN compensation_documents d ON d.impact_process_id = c.impact_process_id
WHERE c.period_key = :period_key;
```

Write / upsert query

```
INSERT INTO document_competitors
(doc_no, competitor_code, name_th, branch_th, opened_date, closed_date, source_system, updated_at)
VALUES (:doc_no, :competitor_code, :name_th, :branch_th, :opened_date, :closed_date, 'ALLMAP', CURRENT_TIMESTAMP)
ON CONFLICT (doc_no, competitor_code)
DO UPDATE SET name_th = EXCLUDED.name_th, branch_th = EXCLUDED.branch_th,
opened_date = EXCLUDED.opened_date, closed_date = EXCLUDED.closed_date,
updated_at = CURRENT_TIMESTAMP;

DELETE FROM document_competitors dc
WHERE dc.doc_no = :doc_no
AND dc.source_system = 'ALLMAP'
AND NOT EXISTS (
  SELECT 1
  FROM fgi_impact_competitors src
  JOIN compensation_documents d ON d.impact_process_id = src.impact_process_id
  WHERE d.doc_no = dc.doc_no
  AND src.period_key = :period_key
  AND src.competitor_code = dc.competitor_code
);
```

5.94 Target Node Implementation

โครงสร้างนี้ระบุ service/repository เฉพาะงานและต้อง implement ตาม SQL, transaction, idempotency และ security contract ด้านบน โดยทุกชั้นต้องคืน metrics สำหรับ reconcile และ run history

```
export async function runLddBeJob7Synccompetitortodocument(ctx, services) {
  const run = await services.jobRuns.acquire({
    jobNo: "7", period: ctx.period, triggeredBy: ctx.triggeredBy
  });

  try {
    ctx = { ...ctx, runId: run.id, repository: services.documentCompetitorRepository };
    const step1 = await services.loadLatestDocumentCompetitors(ctx, undefined);
    const step2 = await services.upsertDocumentCompetitors(ctx, step1);
    const step3 = await services.recordInternalCompetitorSync(ctx, step2);
    const step4 = await services.reconcileDocumentCompetitors(ctx, step3);
    const result = step4;
    await services.jobRuns.finish(run.id, "SUCCESS", result.metrics);
    return { runId: run.id, status: "SUCCESS", ...result };
  } catch (error) {
    await services.jobRuns.finish(run.id, "FAILED", {
      errorCode: error.code ?? "JOB_FAILED",
      errorMessage: error.message
    });
    throw error;
  }
}
```

6. Button / User Action Mapping

Action	Trigger	API / Service	Expected Result
รันตามตารางเวลา	CRON	scheduler → runner (job 7)	อ่าน cron/พารามิเตอร์จาก backend config
รันนอกรอบ (manual/rerun)	CLI	CLI/ops runbook → runner (job 7)	guard ไม่ให้รันซ้อนด้วย distributed lock
แก้พารามิเตอร์/เปิด-ปิด job	CONFIG	แก้ backend config แล้ว deploy	ไม่มี endpoint และไม่มีหน้าจอบทความ — หน้า Flow Batch Job เป็น reference อย่างเดียว (2026-08-06)
ตรวจสอบการรัน	LOG	application log (structured)	ไม่มีตาราง job_run_histories แล้ว · ไฟล์/ACK ดูที่ interface_transactions

7. API Contract

8. Reference DB Mapping (No Database Page Work)

ส่วนนี้เป็นข้อมูลอ้างอิงสำหรับการ implement API/Job เท่านั้น ไม่ใช้งานสร้างหน้า Database, ไม่ใช้งานออกแบบ DB page และไม่ถูกนับเป็น deliverable แยกของ FE/BE

Table / Object	R/W	Usage
fgi_impact_competitors	R	ข้อมูลคู่แข่งล่าสุดจาก Job 3
compensation_documents	R	หา doc_no จาก impact_process_id
document_competitors	W	บันทึกคู่แข่งเข้าเอกสารโดยตรง
interface_transactions	W	tracking ภายใน type=INTERNAL_DB_WRITE

9. Skeleton Code (Batch Job 7)

9.1 ไฟล์ที่ต้องสร้าง (Job 7)

โครงไฟล์ของ Job 7 (document.service.syncCompetitors เดิม) วางใต้ src/batch/sbpgi/ ของ store-backend โดยใช้ convention เดียวกับ module อื่นๆ: inject custom provider DATA_SOURCE แล้วยิง raw SQL, repository ประกาศเป็น factory provider ที่ใช้ token string, entity อยู่ใน src/entities/

หมายเหตุสำคัญ — src/batch/* ทั้งหมดเป็นของใหม่ที่ยังไม่มีใน store-backend: ปัจจุบัน repo ไม่มีโฟลเดอร์ src/batch เลย และแม้จะติดตั้ง @nestjs/schedule ไว้แล้วก็ยังไม่มี @Cron/@Interval แม้แต่จุดเดียว ดังนั้น runner.ts / scheduler.ts / cli.js / job-failure.notifier.ts คือ งานตั้งต้นของ Phase แรก ที่ต้องสร้างเองทั้งหมด พร้อม register ScheduleModule.forRoot() ใน app.module.ts — ไม่ใช่ของเดิมที่ reuse ได้

Path	หน้าที่
src/batch/sbpgi/job-7-sync-competitor-to-document/job-7-sync-competitor-to-document.job.ts	คลาส SyncCompetitorToDocumentJob — run(ctx) เรียงตาม flow ของ Job 7 ที่ละขั้น, ครอบ transaction, จบด้วย structured log
src/batch/sbpgi/job-7-sync-competitor-to-document/job-7-sync-competitor-to-document.service.ts	คลาส SyncCompetitorToDocumentService — logic ต่อชั้น (อ่าน/parse/คำนวณ/เขียน) + repository token ที่ inject จาก DATA_SOURCE
src/batch/sbpgi/job-7-sync-competitor-to-document/job-7-sync-competitor-to-document.config.ts	คลาส SbpgiJob7Config (แบบเดียวกับ src/config/app.config.ts — โปรเจกต์นี้ไม่ใช่ registerAs) — cron และพารามิเตอร์ทั้ง 3 ตัวของ Job 7 อ่านจาก env/config file (ไม่มีตาราง job_configs)
src/batch/sbpgi/job-7-sync-competitor-to-document/job-7-sync-competitor-to-document.module.ts	NestJS module ผูก job + service + repository provider (factory token string) เข้ากับ DatabaseModule
src/batch/runner.ts	ตัวรันกลาง: resolve job ตาม jobNo, กั้นรันซ้อนด้วย advisory lock, จับ error → แจ้งเตือน, เขียน structured log สรุป (ใช้รวมทั้ง 11 job)
src/batch/scheduler.ts	ลงทะเบียน cron จาก config (SBPGI_JOB7_CRON = 30 17 7-31 * *) และรองรับสิ่งรันนอกกรอบผ่าน CLI/runbook
src/batch/job-failure.notifier.ts	ส่งอีเมลแจ้งผู้ดูแลเมื่อ job ล้มเหลว ผ่าน EmailLibService ของ @gsoft-sbp/email-lib (log ลง email_sent ให้อัตโนมัติ)

9.2 Config Schema ของ Job 7 (backend config / env)

cron ปัจจุบันของ Job 7 คือ 30 17 7-31 * * (วันที่ 7-31 เวลา 17:30) — ประกาศเป็น SBPGI_JOB7_CRON และอ่านตอน bootstrap ของ scheduler.ts; ถ้า enabled=false scheduler ต้องไม่ลงทะเบียน cron ของ job นี้

```
// src/batch/sbpgi/job-7-sync-competitor-to-document/job-7-sync-competitor-to-document.config.ts
// convention จริงของ store-backend คือคลาส config ('src/config/app.config.ts' ที่ export ผ่าน
// 'AppConfigModule' แบบ @Global แล้วอ่าน process.env ตรงๆ) — โปรเจกต์นี้ **ไม่ได้ใช้ registerAs**
// แม้แต่จุดเดียว จึงประกาศเป็นคลาสให้รีวิว/ทดสอบเหมือน config ตัวอื่น
import { Injectable } from '@nestjs/common';

// TODO: Job 7 ไม่มีตาราง job_configs และไม่มี Job Admin API แล้ว (ตัดสินใจ 2026-08-06)
// TODO: ค่าทุกตัวอ่านจาก env/config file ของ backend เท่านั้น — เปลี่ยนค่า = แก้ config แล้ว deploy
export interface Job7Config {
```

```

/** เปิด/ปิด job รอบถัดไปโดยไม่ต้อง deploy โค้ด */
enabled: boolean;
/** cron ของ job นี้ (อ่านตอน bootstrap ของ scheduler.ts) */
cron: string;
/** กำหนดการรัน (Cron) — ใช้รอบเดิม แต่ปลายทางเป็น DB ภายใน */
cron: string;
/** Target table — upsert ด้วย doc_no / competitor_code / source_system=ALM */
targetTable: string;
/** เงื่อนไขเลือกข้อมูล — คง business rule เดิม */
condition: string;
/** ผู้รับอีเมลเมื่อ job ล้มเหลว — เก็บเป็น string คั่น comma ให้ตรง signature ของ
  'EmailLibService.sendMail({ mailTo })' ที่รับ string ไม่ใช่ string[] */
mailTo: string;
}

@Injectable()
export class SbpqiJob7Config implements Job7Config {
  // TODO: ยืนยันค่า default ทุกตัวกับ Ops ก่อนขึ้น production (ไม่มีหน้าจอกำหนดค่าแล้ว)
  enabled = (process.env.SBPGI_JOB7_ENABLED ?? 'true') === 'true';
  cron = process.env.SBPGI_JOB7_CRON ?? '30 17 7-31 * *';
  cron = process.env.SBPGI_JOB7_CRON ?? '30 17 7-31 * *'; // TODO: แก่ผ่าน env/config file แล้ว deploy
  targetTable = process.env.SBPGI_JOB7_TARGET_TABLE ?? 'document_competitors'; // TODO: ค่าคงที่ทางธุรกิจ — เปลี่ยนต้องผ่านการอนุมัติ
  condition = process.env.SBPGI_JOB7_CONDITION ?? 'งวดคู่แข่งล่าสุดต่อรัน + forecast เริ่มต้น + ยังไม่ sync'; // TODO: ค่าคงที่ทางธุรกิจ —
  เปลี่ยนต้องผ่านการอนุมัติ
  mailTo = process.env.SBPGI_JOB7_MAIL_TO ?? ''; // TODO: ผู้รับอีเมลแจ้ง error คั่นด้วย comma (เดิม: ส่ง error ผ่าน Notification Service กลางเมื่อ
  sync ล้มเหลว)
}

// TODO: เพิ่ม SbpqiJob7Config ใน providers/exports ของ AppConfigModule (@Global) เหมือน AppConfig

```

9.3 Job Class — `run(ctx)` ของ Job 7 ที่ละชั้นตามผัง

9.3.1 สัญญาของชั้นกลาง (`runner.ts`) + โครง service ของ Job 7

job class อ้าง JobRunContext / JobRunResult / JobState / JobFailedError — ทั้งหมดนิยาม ครั้งเดียวใน src/batch/runner.ts (ไฟล์รวมของทุก job ให้ merge ไม่ใช่เขียนทับ) และ service ต้องมี method ครบตามตารางชั้นตอนด้านล่าง มิฉะนั้น job class จะเรียก method ที่ไม่มีอยู่

```

// src/batch/runner.ts — สัญญาของทุก job (ประกาศครั้งเดียว ใช้ร่วมทั้ง 11 ฉบับ)

export interface JobRunContext {
  jobNo: string;
  period: string; // YYYYMM ของงวดที่รัน
  triggeredBy: string; // 'CRON' | userId ที่สั่งรันรอบรอบ
  params?: Record<string, string>;
}

export interface JobRunResult {
  event: 'job.finish';
  jobNo: string;
  jobName: string;
  status: 'SUCCESS' | 'SKIPPED' | 'SKIPPED_LOCKED' | 'FAILED';
  period: string;
  output: string;
  read: number; written: number; skipped: number; rejected: number;
  durationMs: number;
}

/** counter + ค่าที่ทุกชั้นของ job ใช้ร่วมกัน (service เป็นผู้สร้างผ่าน createState) */
export interface JobState {
  period: string;
  read: number; written: number; skipped: number; rejected: number;
  // TODO: เพิ่ม field เฉพาะของ job นี้ (เช่น rows ที่อ่านมา, path ไฟล์ที่เขียน)
  [key: string]: unknown;
}

/** error ที่ทำให้ job จบเป็น FAILED และส่งอีเมลแจ้งผู้ดูแล */
export class JobFailedError extends Error {
  constructor(public readonly code: string, message: string) { super(message); }
}

/** ให้ออกจาก transaction เมื่อสาขา NO บอกให้ข้ามงวด/เรคคอร์ด — runner สรุปรเป็น SKIPPED ไม่ใช่ FAILED */
export class JobSkippedError extends Error {}

// SyncCompetitorToDocumentService — method ที่ job class เรียก (1 method ต่อ 1 ชั้นในตารางด้านบน)
import { Inject, Injectable } from '@nestjs/common';
import type { DataSource, EntityManager } from 'typeorm';
import type { JobRunContext, JobState } from '../runner';
export type { JobState };

@Injectable()
export class SyncCompetitorToDocumentService {

```

```

constructor(@Inject('DATA_SOURCE') private readonly dataSource: DataSource) {}

createState(ctx: JobRunContext): JobState {
  return { period: ctx.period, read: 0, written: 0, skipped: 0, rejected: 0 };
}

// อ่านคู่แข่งงวดล่าสุดต่อร้านจาก fgi_impact_competitors
async step02Read(state: JobState, manager?: EntityManager): Promise<void> {
  // TODO: implement
}

// มี compensation_documents ของ impact_process_id แล้ว?
async check03Document(state: JobState): Promise<boolean> {
  return true; // TODO: เงื่อนไขจริงตามผัง
}

// upsert document_competitors
async step04Upsert(state: JobState, manager?: EntityManager): Promise<void> {
  // TODO: implement
}

// บันทึก interface_transactions เป็น INTERNAL_DB_WRITE
async step05WriteFile(state: JobState, manager?: EntityManager): Promise<void> {
  // TODO: implement
}
}

```

9.3.2 `run(ctx)` ของ Job 7

ทุกขั้นใน `run()` ตรงกับ flowchart ของ Job 7 หนึ่งต่อหนึ่ง (decision และ error path รวมอยู่ด้วย) — method ที่ต้อง implement ใน service ตามตารางนี้

ลำดับ	ชนิด	ขั้นตอนจากผัง	Method ที่ต้อง implement	เส้นทาง NO / error
1	start	เริ่ม	<code>createState()</code>	-
2	process	อ่านคู่แข่งงวดล่าสุดต่อร้านจาก <code>fgi_impact_competitors</code>	<code>step02Read()</code>	throw <code>JobFailedError</code> เมื่อทำไม่สำเร็จ
3	decision	มี <code>compensation_documents</code> ของ <code>impact_process_id</code> แล้ว?	<code>check03Document()</code>	[err] คงสถานะรอ sync / log pending
4	process	upsert <code>document_competitors</code>	<code>step04Upsert()</code>	throw <code>JobFailedError</code> เมื่อทำไม่สำเร็จ
5	process	บันทึก <code>interface_transactions</code> เป็น <code>INTERNAL_DB_WRITE</code>	<code>step05WriteFile()</code>	throw <code>JobFailedError</code> เมื่อทำไม่สำเร็จ
6	end	จบ	<code>summarize()</code>	-

```

// src/batch/sbpgi/job-7-sync-competitor-to-document/job-7-sync-competitor-to-document.job.ts
import { Inject, Injectable, Logger } from '@nestjs/common';
import type { DataSource, EntityManager } from 'typeorm';
import { SyncCompetitorToDocumentService, type JobState } from '../job-7-sync-competitor-to-document.service';
// 4 symbol นี้มีใน src/batch/runner.ts (ดูหัวข้อ 9.3.1)
import { JobFailedError, JobSkippedError, JobRunContext, JobRunResult } from '../../runner';

@Injectable()
export class SyncCompetitorToDocumentJob {
  static readonly jobNo = '7';
  private readonly logger = new Logger(SyncCompetitorToDocumentJob.name);

  constructor(
    // TODO: DATA_SOURCE = custom provider ที่ route SELECT/WITH ไป slave pool และ write ไป master
    @Inject('DATA_SOURCE') private readonly dataSource: DataSource,
    private readonly service: SyncCompetitorToDocumentService,
  ) {}

  async run(ctx: JobRunContext): Promise<JobRunResult> {
    const startedAt = Date.now();
    // TODO: state คือ counter (read/written/skipped/rejected) และค่าจาก job7Config
    const state = this.service.createState(ctx);
    try {
      // ขั้นที่ 2: อ่านคู่แข่งงวดล่าสุดต่อร้านจาก fgi_impact_competitors · TODO: dense rank ตามงวดต้นทางของคู่แข่ง
      await this.service.step02Read(state);
      // ขั้นที่ 3 (decision): มี compensation_documents ของ impact_process_id แล้ว?
      const ok03 = await this.service.check03Document(state);
    }
  }
}

```



```

if (!ok03) throw new JobFailedError('JOB7_STEP03', 'คงสถานะรอ sync / log pending');
// === transaction boundary === TODO: DB transaction ครอบการ upsert document_competitors + tracking
await this.dataSource.transaction(async (manager: EntityManager) => {
  // ขั้นที่ 4: upsert document_competitors · TODO: source_system=ALM, ผูก doc_no และ competitor_code
  await this.service.step04Upsert(state, manager);
  // ขั้นที่ 5: บันทึก interface_transactions เป็น INTERNAL_DB_WRITE · TODO: ไม่สร้างไฟล์ BPM06003O
  await this.service.step05WriteFile(state, manager);
});
return this.summarize(state, 'SUCCESS', startedAt);
} catch (error) {
  // TODO: error path ของ Job 7 — ห้าม re-implement การเขียนไฟล์ BPM06003O หรือ SFTP ไป BPM; legacy file เป็น reference เท่านั้น
  this.logger.error(JSON.stringify({ event: 'job.failed', jobNo: '7', period: ctx.period,
    triggeredBy: ctx.triggeredBy, durationMs: Date.now() - startedAt, error: (error as Error).message }));
  // TODO: แจ้งผู้ดูแลผ่าน JobFailureNotifier (หัวข้อ 9.6.1) — runner เป็นผู้เรียกให้
  throw error;
}
}

private summarize(state: JobState, status: JobRunResult['status'], startedAt = Date.now()): JobRunResult {
  // TODO: structured log บรรทัดเดียวจบ — ไม่มีตาราง job_run_histories แล้ว (2026-08-06)
  const summary = {
    event: 'job.finish', jobNo: '7', jobName: 'SyncCompetitorToDocument', status,
    period: state.period, output: 'document_competitors (DB)',
    read: state.read, written: state.written, skipped: state.skipped,
    rejected: state.rejected, durationMs: Date.now() - startedAt,
  };
  this.logger.log(JSON.stringify(summary));
  return summary as JobRunResult;
}
}

```

9.4 การกันรันซ้อนของ Job 7 (PostgreSQL advisory lock)

Job 7 มีข้อควรระวังจาก legacy: ห้าม re-implement การเขียนไฟล์ BPM06003O หรือ SFTP ไป BPM; legacy file เป็น reference เท่านั้น — runner ล็อกด้วย pg_try_advisory_lock ก่อนเริ่มขั้นแรกเสมอ และรอบที่ล็อกไม่ได้ให้จบด้วยสถานะ SKIPPED_LOCKED (ไม่ใช่ FAILED)

```

// src/batch/runner.ts (ส่วนกันรันซ้อน)
import { Inject, Injectable, Logger } from '@nestjs/common';
import type { DataSource } from 'typeorm';

// TODO: ห้ามใช้แถวสถานะ RUNNING ในตารางเป็นตัวกัน (ไม่มีตาราง job_run_histories แล้ว)
// ใช้ PostgreSQL advisory lock ระดับ session แทน — ปลดล็อกอัตโนมัติเมื่อ connection หลุด
export const SBPGI_JOB_LOCK_CLASS_ID = 861000; // namespace ของระบบ SBPGI
export const JOB_LOCK_KEYS: Record<string, number> = { '7': 70 /* TODO: เพิ่มครบทั้ง 11 job */ };

@Injectable()
export class BatchRunner {
  private readonly logger = new Logger(BatchRunner.name);
  constructor(@Inject('DATA_SOURCE') private readonly dataSource: DataSource) {}

  async runExclusive<T>(jobNo: string, fn: () => Promise<T>): Promise<T | { status: 'SKIPPED_LOCKED' }> {
    // TODO: ต้องใช้ QueryRunner (connection เดียวกัน master) — dataSource.query() ของโปรเจกต์นี้
    // route SQL ที่ขึ้นต้นด้วย SELECT ไป slave pool ทำให้ lock ไปตกที่ replica คนละ connection
    const runner = this.dataSource.createQueryRunner('master');
    await runner.connect();
    const objectId = JOB_LOCK_KEYS[jobNo];
    try {
      const [{ locked }] = await runner.query(
        'SELECT pg_try_advisory_lock($1, $2) AS locked',
        [SBPGI_JOB_LOCK_CLASS_ID, objectId],
      );
      if (!locked) {
        // TODO: รอบนี้ข้ามไปเลย ๆ ไม่ถือเป็น error และไม่ต้องส่งอีเมล
        this.logger.warn(JSON.stringify({ event: 'job.skipped.locked', jobNo }));
        return { status: 'SKIPPED_LOCKED' };
      }
    } finally {
      return await fn();
    }
  }
}

```

9.5 Repository / SQL หลักของ Job 7

repository ของ Job 7 ประกาศเป็น factory provider ({provide: 'SYNC_COMPETITOR_TO_DOCUMENT_REPOSITORY', useFactory: (ds) => ds.getRepository(Entity), inject: ['DATA_SOURCE']}) แล้วยิง raw SQL ตามแบบ module ธุรกิจอื่นของ store-backend (schema sps_store มาจาก search_path)

ตาราง	R/W	การใช้งานตามผัง	หมายเหตุ target design
fgi_impact_competitors	R	ข้อมูลคู่แข่งล่าสุดจาก Job 3	เขียน SQL ตรงผ่าน DATA_SOURCE
compensation_documents	R	หา doc_no จาก impact_process_id	เขียน SQL ตรงผ่าน DATA_SOURCE
document_competitors	W	บันทึกคู่แข่งเข้าเอกสารโดยตรง	เขียน SQL ตรงผ่าน DATA_SOURCE
interface_transactions	W	tracking ภายใน type=INTERNAL_DB_WRITE	เขียน SQL ตรงผ่าน DATA_SOURCE

```
-- Job 7 SyncCompetitorToDocument — query หลักที่ต้อง implement
-- TODO: ทุก statement รันผ่าน DATA_SOURCE (SELECT ไป slave, write ไป master) และ
-- write ทั้งหมดต้องอยู่ใน transaction เดียวกันที่ระบุใน 9.3

-- [R] fgi_impact_competitors : ข้อมูลคู่แข่งล่าสุดจาก Job 3
-- TODO: เพิ่มเฉพาะคอลัมน์ที่ job ไซจริง (ห้าม SELECT *) และตรวจว่ามี index รองรับ WHERE นี้
SELECT /* TODO: columns */
FROM fgi_impact_competitors
WHERE impact_year = $1 AND impact_month = $2 -- TODO: ยืนยันชื่อคอลัมน์จาก Database design
ORDER BY /* TODO: คีย์ที่ทำให้ลำดับคงที่ */
LIMIT $3 OFFSET $4; -- TODO: อ่านเป็น chunk กัน memory บวม

-- [R] compensation_documents : หา doc_no จาก impact_process_id
-- TODO: เพิ่มเฉพาะคอลัมน์ที่ job ไซจริง (ห้าม SELECT *) และตรวจว่ามี index รองรับ WHERE นี้
SELECT /* TODO: columns */
FROM compensation_documents
WHERE /* TODO: เงื่อนไขช่วง/สถานที่ job นี้คัดแถว */ 1 = 1
ORDER BY /* TODO: คีย์ที่ทำให้ลำดับคงที่ */
LIMIT $1 OFFSET $2; -- TODO: อ่านเป็น chunk กัน memory บวม

-- [W] document_competitors : บันทึกคู่แข่งเข้าเอกสารโดยตรง
-- TODO: เพิ่มคอลัมน์ payload จริงจาก Database design
INSERT INTO document_competitors
(/* TODO: business key + payload + created_by, created_at */)
VALUES /* TODO: bind params ตามลำดับคอลัมน์ตามบน */
ON CONFLICT (doc_no, competitor_code) -- unique key จริงตาม DDL ของ document_competitors (ห้ามเดา)
DO UPDATE SET /* TODO: คอลัมน์ที่ยอมให้ทับ */
updated_at = NOW(), updated_by = 'JOB7';

-- [W] interface_transactions : tracking ภายใน type=INTERNAL_DB_WRITE
-- TODO: บันทึก ACK ระดับ record ของไฟล์ interface (แทน job_run_histories ที่ยกเลิกไปแล้ว)
INSERT INTO interface_transactions
(run_id, data_name, direction, status, business_key, period_key,
file_name, file_checksum, created_at)
VALUES ($1 /* run_id = correlation id ของรอบรัน Job 7 จาก application log */,
$2 /* TODO: data_name ของ Job 7 */, $3 /* IN|OUT|INTERNAL */, 'READY',
$4 /* business key ของแถว */, $5 /* YYYYMM */, $6, $7, NOW())
ON CONFLICT (data_name, direction, business_key, period_key) DO NOTHING;
```

9.6 การแจ้งเตือนและการรันซ้ำของ Job 7

9.6.1 อีเมลแจ้งผู้ดูแลเมื่อ job ล้มเหลว

ใช้ EmailLibService จาก @gosoft-sbp/email-lib ตัวเดียวกับที่ระบบเดิมใช้ (inform-evaluate / external-audit / statement PTT) — ไม่สร้างกลไกส่งเมลใหม่

```
// src/batch/job-failure.notifier.ts
import { Injectable, Logger } from '@nestjs/common';
// ชื่อ method ของ lib ที่ store-backend เรียกจริงคือ 'sendMail' (ไม่ใช่ sendEmail) และ
// 'mailTo' / 'mailCc' เป็น **string** คั่นด้วย comma — ดู evaluation-process.service.ts,
// external-audit.service.ts, statement.service.ts, inform-evaluate.service.ts, performance.service.ts
import { EmailLibService } from '@gosoft-sbp/email-lib';
import type { JobRunContext } from './runner';

@Injectable()
export class JobFailureNotifier {
  private readonly logger = new Logger(JobFailureNotifier.name);
  // TODO: ใช้ lib อีเมลของระบบเดิม — template อยู่ในตาราง email_template และ log ลง email_sent อัตโนมัติ
  // (ตั้งชื่อ property ว่า mailService ตาม call site เดิมทุกที่ใน store-backend)
  constructor(private readonly mailService: EmailLibService) {}
```

```

async notifyFailure(jobNo: string, ctx: JobRunContext, error: Error): Promise<void> {
  // TODO: ผู้รับของ Job 7 เดิมคือ ส่ง error ผ่าน Notification Service กลางเมื่อ sync ล้มเหลว — ย้ายมาเป็น env SBPGI_JOB7_MAIL_TO
  const recipients = (process.env.SBPGI_JOB7_MAIL_TO ?? "").split(',').map((s) => s.trim()).filter(Boolean);
  if (!recipients.length) {
    this.logger.warn(JSON.stringify({ event: 'job.mail.skipped', jobNo, reason: 'NO_RECIPIENT' }));
    return;
  }
  try {
    await this.mailService.sendMail({
      // TODO: emailId = id ของ template EM-07 (แจ้ง error batch) ในตาราง email_template
      emailId: Number(process.env.SBPGI_JOB_FAIL_EMAIL_TEMPLATE_ID),
      mailTo: recipients.join(','), // signature รับ string ไม่ใช่ string[]
      mailCc: "",
      param: {
        jobNo, jobName: 'SyncCompetitorToDocument',
        jobTitle: 'บันทึกข้อมูลคู่แข่งเขาเอกสาร',
        period: ctx.period, triggeredBy: ctx.triggeredBy,
        output: 'document_competitors (DB)',
        errorMessage: error.message,
        rerunNote: 'idempotent ด้วย doc_no + competitor_code + source_system',
      },
    });
  } catch (mailError) {
    // TODO: ส่งเมลไม่สำเร็จห้ามกลับ error เดิมของ job — log แล้วปล่อยผ่าน
    this.logger.error(JSON.stringify({ event: 'job.mail.failed', jobNo, error: (mailError as Error).message }));
  }
}

```

9.6.2 Checklist การ rerun

- กติกา rerun ของ Job 7: idempotent ด้วย doc_no + competitor_code + source_system
- ขอบเขต transaction ที่ต้องรักษามือรันซ้ำ: DB transaction ครอบคลุมการ upsert document_competitors + tracking
- ความเสี่ยงที่ต้องตรวจก่อน/หลังรันซ้ำ: ห้าม re-implement การเขียนไฟล์ BPM06003O หรือ SFTP ไป BPM; legacy file เป็น reference เท่านั้น
- ตรวจสอบว่ารอบก่อนหน้าไม่ได้ค้าง lock อยู่ (SELECT * FROM pg_locks WHERE locktype = 'advisory') ก่อนสั่งรันนอกรอบ
- สั่งรันนอกรอบผ่าน CLI/runbook เท่านั้น (ไม่มีหน้าจอลงและไม่มี Job Admin API): `node dist/batch/cli.js --job=7 --period=<YYYYMM>`
- หลังรันซ้ำ ตรวจสอบ output document_competitors (DB) และ log บรรทัด `job.finish` ว่า read/written/skipped/rejected ตรงกับที่คาด
- ถักรอบก่อนล้มเหลวกลางทาง ตรวจสอบ interface_transactions ของงวดนั้นว่ามีแถวค้างสถานะ READY/PENDING หรือไม่ ก่อนสั่งรันใหม่

10. Processing Flow

Step	Description
1	เริ่ม
2	อ่านคู่แข่งงวดล่าสุดต่อร้านจาก fgi_impact_competitors (dense rank ตามงวดต้นทางของคู่แข่ง)
3	มี compensation_documents ของ impact_process_id แล้ว? No: คงสถานะรอ sync / log pending
4	upsert document_competitors (source_system=ALM, ผูก doc_no และ competitor_code)
5	บันทึก interface_transactions เป็น INTERNAL_DB_WRITE (ไม่สร้างไฟล์ BPM06003O)
6	จบ

11. Acceptance Criteria

- พารามิเตอร์และ cron อ่านจาก backend config เท่านั้น — เปลี่ยนค่าโดย deploy config ไม่ใช่ผ่าน API/หน้าจอ
- การรันต้องตรวจ enabled flag ใน config และกันรันซ้อนด้วย distributed/advisory lock
- ทุกรอบต้องเขียน application log แบบ structured (เวลา/แถว/ไฟล์/ผล) และ error ต้องส่ง EM-07
- DB/table mapping ใช้เป็น reference สำหรับ implement Job เท่านั้น ไม่ใช่งานสร้างหน้า Database
- รองรับ rerun rule และ risk note ตาม runbook

12. Developer Test Checklist

No	Test
1	รันตามตารางเวลาแล้วผลถูกต้องบน fixture
2	รันนอกรอบผ่าน CLI ได้ผลเดียวกับ cron
3	สั่งรันซ้อนขณะกำลังรัน → runner ปฏิเสธ (lock ทำงาน)
4	แก้ config แล้ว deploy → รอบถัดไปใช้ค่าใหม่
5	job throw error → EM-07 ออก และ log มีบรรทัด error
6	ตรวจสอบผลกระทบตารางตาม R/W mapping reference

13. Unit Test Scope

3 ชั่วโมง (30% ของ implementation 10 ชั่วโมง) · เครื่องมือ: Jest + mock repository/DataSource (ไม่ต่อ DB จริง)

หัวข้อนี้คือ unit test ที่ต้องเขียนคู่กับโค้ด — ต่างจาก *Developer Test Checklist* ซึ่งเป็น scenario ระดับ end-to-end/manual ที่ใช้ตอนตรวจรับ · รายการด้านล่าง derive จาก field/validation, acceptance criteria, endpoint และตารางที่เอกสารนี้เขียน

สิ่งที่ทดสอบ	ประเภท	เกณฑ์ผ่าน
เงื่อนไขเลือกข้อมูล	rule	ใช้กับข้อมูลตัวอย่างแล้วได้ผลตามที่ระบุ — งดคู่แข่งล่าสุดต่อรัน + forecast เริ่มต้น + ยังไม่ sync
business rule	logic	พารามิเตอร์และ cron อ่านจาก backend config เท่านั้น — เปลี่ยนค่าโดย deploy config ไม่ใช่ผ่าน API/หน้าจอ
business rule	logic	การรันต้องตรวจ enabled flag ใน config และกันรันซ้อนด้วย distributed/advisory lock
business rule	logic	ทุกรอบต้องเขียน application log แบบ structured (เวลา/แถว/ไฟล์/ผล) และ error ต้องส่ง EM-07
business rule	logic	DB/table mapping ใช้เป็น reference สำหรับ implement Job เท่านั้น ไม่ใช่งานสร้างหน้า Database
business rule	logic	รองรับ rerun rule และ risk note ตาม runbook
document_competitors, interface_transactions	transaction	จำลอง error กลางทาง แล้วยืนยันว่า rollback ครบ ไม่เหลือแถวค้าง (mock DataSource/QueryRunner)
runner	idempotency	รันซ้ำด้วย fixture เดิมต้องไม่เกิดแถวซ้ำ (ON CONFLICT / business unique key ทำงาน)
runner	lock	เรียกซ้อนขณะกำลังรัน ต้องถูกปฏิเสธด้วย advisory lock

- ทุกเคสต้องรันได้โดยไม่ต่อ DB/บริการภายนอกจริง — mock ที่ขอบ repository/client เสมอ
- ข้อความไทยที่ยืนยันในเทสต้องเป็น verbatim ตาม ข้อกำหนดทางธุรกิจ ห้ามพิมพ์ใหม่
- เกณฑ์ผ่านของ CI: ทุกเคสในตารางนี้มี test จริงและผ่านทั้งหมด

ภาคผนวก: Java Source เดิม

Code ในส่วนนี้คัดจาก Java source เดิมโดยตรงและคงข้อความตามต้นฉบับ ใช้เลขบรรทัดที่ระบุหน้าทุก snippet เพื่อตรวจเทียบ business behavior, SQL, transaction และ error handling

J1. ExportCompetitor.java — lines 9-20

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/ExportCompetitor.java
Original lines	9-20
Responsibility	Legacy main entrypoint for competitor export.

```
public class ExportCompetitor {  
  
    private static ApplicationContext context = new FileSystemXmlApplicationContext("config.xml");  
    private static ExportController exportController = (ExportController) context.getBean("exportController");  
  
    public static void main(String[] args) {  
        LogUtils.info(ExportCompetitor.class,"Start => export IMPACT_COMPETITOR to BPM");  
        exportController.exportCompetitorToBPM();  
        LogUtils.info(ExportCompetitor.class,"End => export IMPACT_COMPETITOR to BPM");  
    }  
  
}
```

J2. ExportController.java — lines 659-670

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/controller/ExportController.java
Original lines	659-670
Responsibility	Query competitor data, generate file content, upload, backup, notification.

```
public void exportCompetitorToBPM() {  
    String fileName = "";  
    Calendar calStart = Calendar.getInstance(Locale.US);  
    EmailTemplateInformStatusBean informBean = new EmailTemplateInformStatusBean();  
    informBean.setSystemCode(FgiConstant.SYSTEM_CODE);  
    informBean.setJobName(FgiConstant.JOB_NAME_EXPORT_COMPETITOR);  
    informBean.setStartDateTime(calStart.getTime());  
    informBean.setImportFileName("");  
    try {  
        configFtp = (FtpBean) context.getBean("fgiExportCompetitor");  
        setValueConfig();  
        List<Map> competitorList = exportService.queryCompetitorToBPM();
```

J2. ExportController.java — lines 749-760

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/controller/ExportController.java
Original lines	749-760
Responsibility	Query competitor data, generate file content, upload, backup, notification.

```
+ "file_name=" + fileName+ "|total line=" +countLine);

exportService.insertConfirmReceiveData(confirmReceiveDataList);

// upload to server EAI
FtpUtils.uploadSFTPFile(ftpEaiServerIpBpm, ftpEaiUserIdBpm, ftpEaiPasswordBpm, 22, null, 0, ftpSourcePath, ftpDestinationPath, ftpFileNameArray);////////
FileUtils.moveBackupFile(fileExport, fileBackup);
informBean.setStatus(FgiConstant.JOB_STATUS_SUCCESS);
    return new Boolean("true");
} catch (Exception ex){
LogUtils.error(getClass(),ex);
LogUtils.info(getClass(),"===== file isDelete = " + FileUtils.deleteFile(fileExport));
```

J3. ExportJdbc.java — lines 1596-1607

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	1596-1607
Responsibility	Query latest competitor rows eligible for export.

```
public List<Map> queryCompetitorToBPM(){
StringBuilder sql = new StringBuilder();
sql.append(" select * ");
sql.append(" from ( ");
sql.append(" select fic.impact_competitor_id, fic.storecode_i, fic.compet_id, fic.name_th, fic.name, ");
sql.append(" fic.branch_th, fic.zone_code, fic.subzone_code, fic.open_date, fic.close_date, ");
sql.append(" op.last_compensate_month as month, op.last_compensate_year as year, ");
sql.append(" dense_rank() over(partition by fic.storecode_i order by fic.year desc, fic.month desc) as rank_period, ");
sql.append(" 'system' as create_by, fic.create_date, op.impact_process_id ");
sql.append(" from fgi_impact_competitor fic ");
sql.append(" inner join fgi_impact_store_on_process op on op.flag_action in ('"+FgiConstant.FLAG_ACTION_ACTION+"', '"+FgiConstant.FLAG_VERIFY_WAIT+"') ");
sql.append(" and op.datasource = 'ALM' ");
```

J3. ExportJdbc.java — lines 1617-1628

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	1617-1628
Responsibility	Query latest competitor rows eligible for export.

```
sql.append(" and not exists ( ");
sql.append(" select rd.transaction_pk ");
sql.append(" from fgi_confirm_receive_data rd ");
sql.append(" inner join fgi_impact_competitor fic1 on rd.data_name = '"+FgiConstant.DATA_IMPACT_COMPETITOR+"' ");
sql.append(" and fic1.impact_competitor_id = rd.transaction_pk ");
sql.append(" where fic1.storecode_i = a.storecode_i ");
sql.append(" and rd.month = a.month ");
sql.append(" and rd.year = a.year ");
List<Map> impactCompetitorList = jdbcTemplate.queryForList(sql.toString());
LogUtils.info(getClass(),"query FGI_IMPACT_COMPETITOR for export: "+impactCompetitorList.size());
return impactCompetitorList;
}
```